

CIÊNCIA DE DADOS | BANCO E ARMAZÉM DE DADOS

Relatório Técnico

Opção 1: E-commerce Olist Completo

Mini Data Warehouse com DuckDB

Aluno: José Luiz Lopes

Avaliação P02

Data: 03/06/2026

Sumário

Sumário	2
1. Domínio e Dataset	3
1.1. Perguntas a responder	3
1.2. Tecnologia	4
1.3. Repositório (Github)	4
1.4. Organização do Código	4
2. Modelagem Dimensional (30 pts)	5
2.1. Diagrama do Modelo Estrela	5
2.2. Arquivos e Colunas de Dados	5
2.3. Tabelas e Colunas do Modelamento Dimensional	7
3. Pipeline ETL (25 pts)	9
3.1. Script 00 — Setup e Staging	9
3.2. Script 01 — Camada OLTP	10
3.3. Script 02 — Modelo do Data Warehouse	12
3.4. Script 03 — ETL: Carga do DW	15
3.5. Validações	17
4. Consultas Analíticas (20 pts)	18
4.1. Script 04 — Consultas Analíticas	18
5. Visualizações (15 pts)	24
5.1. Geração dos Gráficos	24
5.2. Gráficos Gerados	27
5.3. Gráfico 1 - Evolução de Vendas	27
5.4. Gráfico 2 - Top 10 Produtos	28
5.5. Gráfico 3 - Correlação Ticket Médio vs Avaliação	28
5.6. Gráfico 4 - Dashboard	29
6. Performance e Otimização (10 pts)	29
6.1. Script 05 — Otimização	29
7. GitHub - README	35

1. Domínio e Dataset

Conforme proposta da atividade de avaliação, foi escolhida a **opção 1** com os arquivos de data/Olist/ para poder explorar o dataset completo com os 9 (nove) arquivos de dados.

“Opção 1: Vendas e Logística”

- **Dataset:** *Brazilian E-Commerce Olist (vocês já conhecem!)*
- **Onde baixar:**
 - Kaggle: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
 - OU usar os arquivos que já temos em data/olist/

1.1. Perguntas a responder

"1) Qual é a evolução mensal do faturamento total, quantidade de itens vendidos e do ticket médio da plataforma?"

- Requisitos Atendidos: Análise Temporal e KPI.
- Por que faz sentido para o negócio: Permite monitorar a saúde financeira e o crescimento mês a mês da Olist, identificando sazonalidades.

"2) Quais são as TOP 10 categorias de produtos que geraram maior receita acumulada, e qual a participação percentual de cada uma no total geral?"

- Requisitos Atendidos: Ranking / TOP N, Agregação Multidimensional (Perspectiva de Produto).
- Por que faz sentido para o negócio: Identifica o "core business" em termos de catálogo. Permite focar esforços de marketing e estoque nos produtos que realmente movem a receita.

"3) Quais são os 5 estados (UF) dos compradores com maior ticket médio e como se comporta a nota média de avaliação dos vendedores instalados nesses estados?"

- Requisitos Atendidos: Agregação Multidimensional (Perspectiva Geográfica do Cliente vs. Vendedor) e Ranking.
- Por que faz sentido para o negócio: Cruza a localização de quem compra mais caro com a reputação de quem vende naquela região, ajudando a traçar estratégias de logística e satisfação do cliente.

"4) Qual é a taxa de retenção/recompra mensal dos clientes que fizeram sua primeira compra na plataforma (Análise de Cohort por mês de aquisição)?"

- Requisitos Atendidos: Análise de Cohort / Retenção.
- Por que faz sentido para o negócio: Mede a fidelidade no e-commerce. Clientes que compram apenas uma vez e nunca mais geram um Custo de Aquisição de Cliente (CAC) muito alto. Esta consulta agrupa os clientes pelo mês da primeira compra e conta quantos voltaram nos meses seguintes.

"5) Como os vendedores da plataforma se distribuem na classificação da Curva ABC baseada no faturamento total gerado por cada um?"

- Requisitos Atendidos: Ranking avançado, Funções de Janela (*Window Functions*) e Regra de Negócio Complexa.

- Por que faz sentido para o negócio: Segmenta os parceiros comerciais. Permite descobrir qual a fatia minoritária de vendedores (Classe A) que é responsável por 80% do faturamento da Olist para criar programas de retenção específicos para eles.

1.2. Tecnologia

O pipeline de dados foi desenvolvido utilizando o **DuckDB**, um banco de dados analítico e colunar de alta performance, ideal para processamento local de grandes volumes de dados. A engenharia de dados e as transformações de ETL foram implementadas integralmente por meio da linguagem **SQL nativa do DuckDB**, garantindo consistência, integridade referencial e velocidade nas consultas. Para a camada de visualização e geração de relatórios analíticos, utilizou-se a linguagem Python com as bibliotecas **Plotly e Matplotlib**, integradas ao ambiente de desenvolvimento **VS Code**, enquanto o ciclo de vida do código e o versionamento do projeto foram gerenciados estrategicamente via **Git**.

1.3. Repositório (Github)

O projeto utiliza um repositório no GitHub para gerenciar o versionamento dos scripts SQL e Python sob os princípios de DataOps. O fluxo de trabalho adota commits semânticos para garantir a rastreabilidade das alterações e o isolamento de códigos estáveis na ramificação principal (main). Adicionalmente, o uso do arquivo gitignore impede o envio de dados binários locais (como o demo.duckdb), mantendo o repositório leve, seguro e documentado via README.md para garantir a total reprodutibilidade do Data Warehouse.

https://github.com/joseluiz1990/Projeto_DW_Olist/tree/main

1.4. Organização do Código

```
dw-olist\
  data\olist\                                <- CSVs de entrada
    olist_customers_dataset.csv
    olist_geolocation_dataset.csv
    olist_order_items_dataset.csv
    olist_order_payments_dataset.csv
    olist_order_payments_dataset.csv
    olist_orders_dataset.csv
    olist_products_dataset.csv
    olist_sellers_dataset.csv
    product_category_name_translation.csv
  graficos\                                  <- 3 gráficos PNG e 1 HTML
    grafico_1_evolucao.png
    grafico_2_top_produtos
    grafico_3_dispersao
    dashboard
  scripts\                                   <- Pipeline SQL (executar em ordem)
    00_setup_duckdb.sql                     <- Cria schemas e views de staging
    01_oltp.sql                             <- Cria tabelas OLTP normalizadas
    02_dw_model.sql                         <- Cria o modelo do DW (SCD2)
    03_etl_load.sql                         <- Executa o ETL
    04_analytics.sql                        <- Consultas analíticas
    05_perf.sql                             <- Otimização com tabela agregada
  gerar_graficos.py                         <- Gera 3 gráficos PNG e 1 HTML
  run_all.ps1                              <- Pipeline completo (PowerShell)
  run_all.cmd                              <- Pipeline completo (CMD)
  duckdb.exe                               <- DuckDB CLI para Windows
  demo.duckdb                              <- Banco gerado após rodar o pipeline
```

2. Modelagem Dimensional (30 pts)

2.1. Diagrama do Modelo Estrela



2.2. Arquivos e Colunas de Dados

Esta seção apresenta o mapeamento estrutural dos dados brutos que compõem o ecossistema do Olist E-Commerce. O fluxo operacional foi segmentado em duas etapas iniciais distintas:

Camada de Staging (00_setup_duckdb.sql): Responsável pelo processo de ingestão direta dos arquivos de origem, realizando a carga rápida e espelhamento dos dados para tabelas temporárias (stg_), garantindo o isolamento dos dados brutos.

Camada OLTP (01_oltp.sql): Etapa onde as colunas mapeadas abaixo são lidas, tipadas e normalizadas em um modelo transacional relacional (oltp.), servindo como a fundação limpa para a posterior alimentação do Data Warehouse (DW).

A tabela a seguir detalha os arquivos de origem utilizados, seus respectivos formatos e o dicionário de colunas processadas pelo pipeline:

Tabela de Mapeamento de Arquivos de Origem

Nome Arquivo	Tipo	Colunas
olist_customers_dataset	csv	customer_id, "customer_unique_id", "customer_zip_code_prefix", "customer_city", "customer_state"
olist_geolocation_dataset	csv	geolocation_zip_code_prefix, "geolocation_lat", "geolocation_lng", "geolocation_city", "geolocation_state"
olist_order_items_dataset	csv	order_id, "order_item_id", "product_id", "seller_id", "shipping_limit_date", "price", "freight_value"
olist_order_payments_dataset	csv	order_id, "payment_sequential", "payment_type", "payment_installments", "payment_value"
olist_order_reviews_dataset	csv	review_id, "order_id", "review_score", "review_comment_title", "review_comment_message", "review_creation_date", "review_answer_timestamp"
olist_orders_dataset	csv	order_id, "customer_id", "order_status", "order_purchase_timestamp", "order_approved_at", "order_delivered_carrier_date", "order_delivered_customer_date", "order_estimated_delivery_date"
olist_products_dataset	csv	product_id, "product_category_name", "product_name_lenght", "product_description_lenght", "product_photos_qty", "product_weight_g", "product_length_cm", "product_height_cm", "product_width_cm"

olist_sellers_dataset	csv	seller_id, "seller_zip_code_prefix", "seller_city", "seller_state"
product_category_name_translation	csv	product_category_name, product_category_name_english

2.3. Tabelas e Colunas do Modelamento Dimensional

Esta seção detalha a estrutura do Data Warehouse (DW) sob o esquema de modelagem estrela (Star Schema). Nesta etapa analítica, os dados que antes estavam normalizados no ambiente OLTP são transformados, consolidados e distribuídos em tabelas de dimensões e fatos, otimizando o repositório para consultas de alta performance e suporte à decisão.

O ciclo de vida dessas estruturas é governado por três scripts principais:

Modelagem (02_dw_model.sql): Responsável por criar o schema dw, definir os tipos de dados e aplicar as restrições de integridade referencial (Chaves Primárias e Estrangeiras).

Carga Analítica (03_etl_load.sql): Script de ETL encarregado de povoar as dimensões, gerar as chaves substitutas (Surrogate Keys) por meio de técnicas de histórico SCD Tipo 2, e consolidar as métricas analíticas na tabela fato.

Otimização (05_perf.sql): Etapa complementar onde são gerados índices secundários e criadas estruturas agregadas para acelerar a camada de visualização.

A tabela a seguir apresenta o dicionário de dados do modelo dimensional e os scripts diretamente envolvidos no gerenciamento de cada entidade:

Tabela de Mapeamento do Modelo Dimensional (Data Warehouse)

Nome Tabela	Significado	Tipo	Colunas
dim_geolocation		Dimensão	zip_code_prefix (PK), geolocation_lat, geolocation_lng, city, state
dim_date	Quando é feita a venda?	Dimensão	date_key (PK), full_date, year, month, month_name, quarter, day_of_month, day_name
dim_customer	Quem compra?	Dimensão	customer_key (PK), customer_id , customer_unique_id, zip_code_prefix, city, state, start_date (SCD2), end_date (SCD2),

			is_current (SCD2)
dim_product	Qual produto é vendido?	Dimensão	product_key (PK) product_id, category_name_pt, category_name_en, name_length, description_length, photos_qty, weight_g, length_cm, height_cm , width_cm, is_current
dim_seller	Quem vende?	Dimensão	seller_key, seller_id, zip_code_prefix, city, state
fact_sales	Quais são os resultados das vendas?	Fato	order_id , order_item_id, payment_sequential, Chaves Estrangeiras date_key (FK), customer_key (FK), product_key (FK), seller_key (FK), quantity, price, freight_value, revenue, payment_type, payment_installments, payment_value, review_score, shipping_limit_date,

Tratamento de Histórico (SCD Tipo 2): As dimensões dim_customer e dim_product possuem os atributos start_date, end_date e is_current. Isso indica que o pipeline está preparado para rastrear mudanças históricas de atributos dos clientes (como mudança de endereço) e dos produtos ao longo do tempo, sem sobrescrever dados do passado.

Granularidade da Fato: A tabela fact_sales opera na granularidade do item de cada pedido (order_id + order_item_id), o que permite análises altamente detalhadas de faturamento cruzado, performance de frete e comportamento de categorias de produtos por período temporal.

3. Pipeline ETL (25 pts)

3.1. Script 00 — Setup e Staging

```
# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/00_setup_duckdb.sql"

-- =====
-- SCRIPT: 00_staging.sql
-- ETAPA: 1. STAGING AREA
-- DESCRIÇÃO: Criação de views temporárias que lêem diretamente os 9 arquivos
--             CSV do dataset Olist utilizando a detecção automática do DuckDB.
--             Não são aplicadas transformações ou limpezas nesta camada.
-- INDEMPOTÊNCIA: Utiliza 'CREATE OR REPLACE VIEW' para permitir reexecução.
-- =====

-- Apaga o schema antigo e TUDO o que estiver dentro dele (tabelas e chaves)
DROP SCHEMA IF EXISTS staging CASCADE;

-- Cria o schema novamente do zero, totalmente limpo
CREATE SCHEMA staging;

-- 1. View para o cadastro de clientes
CREATE OR REPLACE VIEW staging.stg_customers AS
SELECT * FROM read_csv_auto('data/olist/olist_customers_dataset.csv');

-- 2. View para a geolocalização e coordenadas dos CEPs
CREATE OR REPLACE VIEW staging.stg_geolocation AS
SELECT * FROM read_csv_auto('data/olist/olist_geolocation_dataset.csv');

-- 3. View para os itens contidos em cada pedido
CREATE OR REPLACE VIEW staging.stg_order_items AS
SELECT * FROM read_csv_auto('data/olist/olist_order_items_dataset.csv');

-- 4. View para os métodos e detalhes de pagamento dos pedidos
CREATE OR REPLACE VIEW staging.stg_order_payments AS
SELECT * FROM read_csv_auto('data/olist/olist_order_payments_dataset.csv');

-- 5. View para as avaliações e notas dadas pelos clientes
CREATE OR REPLACE VIEW staging.stg_order_reviews AS
SELECT * FROM read_csv_auto('data/olist/olist_order_reviews_dataset.csv');

-- 6. View para o cabeçalho e status dos pedidos
CREATE OR REPLACE VIEW staging.stg_orders AS
SELECT * FROM read_csv_auto('data/olist/olist_orders_dataset.csv');

-- 7. View para o cadastro técnico dos produtos
CREATE OR REPLACE VIEW staging.stg_products AS
SELECT * FROM read_csv_auto('data/olist/olist_products_dataset.csv');

-- 8. View para o cadastro de vendedores parceiros (marketplace)
CREATE OR REPLACE VIEW staging.stg_sellers AS
SELECT * FROM read_csv_auto('data/olist/olist_sellers_dataset.csv');

-- 9. View para a tabela de tradução dos nomes das categorias de produtos
CREATE OR REPLACE VIEW staging.stg_category_translation AS
SELECT * FROM read_csv_auto('data/olist/product_category_name_translation.csv');

-- =====
-- VALIDAÇÃO DA CAMADA DE STAGING
-- Consultas rápidas para garantir que as views estão a aceder aos ficheiros
-- =====
SELECT 'stg_orders' AS tabela, COUNT(*) AS total_linhas FROM staging.stg_orders
```

```
UNION ALL
SELECT 'stg_order_items', COUNT(*) FROM staging.stg_order_items;
```

VALIDAÇÃO DA CAMADA DE STAGING

Consultas rápidas para garantir que as views estão a aceder aos ficheiros

```
PS C:\Users\josel\data\dw-atividade-olist-sample> .\duckdb.exe demo.duckdb -c ".read scripts/00_setup_duckdb.sql"
```

tabela varchar	total_linhas int64
stg_orders	99441
stg_order_items	112650

3.2. Script 01 — Camada OLTP

```
# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/01_oltp.sql"

-- =====
-- SCRIPT: 01_oltp.sql
-- ETAPA: 2. CAMADA OLTP (OPERACIONAL)
-- DESCRIÇÃO: Lê as views de staging e cria tabelas físicas estruturadas.
--             Aplica a remoção de duplicatas e o tratamento de nulos.
-- IDEMPOTÊNCIA: Utiliza 'DROP TABLE IF EXISTS' antes de cada criação.
-- =====

-- Apaga o schema antigo e TUDO o que estiver dentro dele (tabelas e chaves)
DROP SCHEMA IF EXISTS oltp CASCADE;

-- Cria o schema novamente do zero, totalmente limpo
CREATE SCHEMA oltp;

-- =====
-- 1. TABELA: oltp.customers
-- =====

CREATE TABLE oltp.customers AS
SELECT DISTINCT
    customer_id,
    customer_unique_id,
    customer_zip_code_prefix AS zip_code_prefix,
    COALESCE(customer_city, 'Não Informado') AS city,
    COALESCE(customer_state, 'ND') AS state
FROM staging.stg_customers;

-- =====
-- 2. TABELA: oltp.products
-- =====

CREATE TABLE oltp.products AS
SELECT DISTINCT
    p.product_id,
    COALESCE(t.product_category_name_english, p.product_category_name, 'Não
Informado') AS category_name_en,
    COALESCE(p.product_category_name, 'Não Informado') AS category_name_pt,
    COALESCE(p.product_name_lenght, 0)::INT AS name_length,
    COALESCE(p.product_description_lenght, 0)::INT AS description_length,
    COALESCE(p.product_photos_qty, 0)::INT AS photos_qty,
    COALESCE(p.product_weight_g, 0)::INT AS weight_g,
    COALESCE(p.product_length_cm, 0)::INT AS length_cm,
    COALESCE(p.product_height_cm, 0)::INT AS height_cm,
```

```

        COALESCE(p.product_width_cm, 0)::INT AS width_cm
FROM staging.stg_products p
LEFT JOIN staging.stg_category_translation t
    ON p.product_category_name = t.product_category_name;

-- -----
-- 3. TABELA: oltp.sellers
-- -----

CREATE TABLE oltp.sellers AS
SELECT DISTINCT
    seller_id,
    seller_zip_code_prefix AS zip_code_prefix,
    COALESCE(seller_city, 'Não Informado') AS city,
    COALESCE(seller_state, 'ND') AS state
FROM staging.stg_sellers;

-- -----
-- 4. TABELA: oltp.geolocation (Agrupada para evitar chaves duplicadas no CEP)
-- -----

CREATE TABLE oltp.geolocation AS
SELECT
    geolocation_zip_code_prefix AS zip_code_prefix,
    AVG(geolocation_lat) AS geolocation_lat,
    AVG(geolocation_lng) AS geolocation_lng,
    ANY_VALUE(geolocation_city) AS city,
    ANY_VALUE(geolocation_state) AS state
FROM staging.stg_geolocation
GROUP BY geolocation_zip_code_prefix;

-- -----
-- 5. TABELA: oltp.orders (Cabeçalho de Pedidos)
-- -----

CREATE TABLE oltp.orders AS
SELECT DISTINCT
    order_id,
    customer_id,
    COALESCE(order_status, 'Não Informado') AS order_status,
    order_purchase_timestamp,
    order_approved_at,
    order_delivered_carrier_date,
    order_delivered_customer_date,
    order_estimated_delivery_date
FROM staging.stg_orders;

-- -----
-- 6. TABELA: oltp.order_items (Itens de Pedidos)
-- -----

CREATE TABLE oltp.order_items AS
SELECT
    order_id,
    order_item_id::INT AS order_item_id,
    product_id,
    seller_id,
    shipping_limit_date,
    price::NUMERIC AS price,
    freight_value::NUMERIC AS freight_value
FROM staging.stg_order_items;

-- -----
-- 7. TABELA: oltp.order_payments (Pagamentos)
-- -----

```

```

CREATE TABLE oltp.order_payments AS
SELECT
    order_id,
    payment_sequential::INT AS payment_sequential,
    COALESCE(payment_type, 'Não Informado') AS payment_type,
    COALESCE(payment_installments, 1)::INT AS payment_installments,
    payment_value::NUMERIC AS payment_value
FROM staging.stg_order_payments;

-----

-- 8. TABELA: oltp.order_reviews (Avaliações)
-----

CREATE TABLE oltp.order_reviews AS
SELECT
    review_id,
    order_id,
    COALESCE(review_score, 0)::INT AS review_score,
    review_comment_title,
    review_comment_message,
    review_creation_date,
    review_answer_timestamp
FROM staging.stg_order_reviews;

-----

-- VALIDAÇÕES DA CAMADA OLTP
-- Verifica se as tabelas físicas foram criadas com volumetria consistente
-----
SELECT 'oltp.orders' AS tabela, COUNT(*) AS total_linhas FROM oltp.orders
UNION ALL
SELECT 'oltp.order_items', COUNT(*) FROM oltp.order_items
UNION ALL
SELECT 'oltp.products', COUNT(*) FROM oltp.products;

```

VALIDAÇÕES DA CAMADA OLTP

Verifica se as tabelas físicas foram criadas com volumetria consistente

```
PS C:\Users\josel\data\dw-atividade-olist-sample> .\duckdb.exe demo.duckdb -c ".read scripts/01_oltp.sql"
```

tabela varchar	total_linhas int64
oltp.orders	99441
oltp.order_items	112650
oltp.products	32951

3.3. Script 02 — Modelo do Data Warehouse

Define a estrutura do DW com Esquema Estrela: dimensões com SCD Tipo 2 e tabela fato.

```

# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/02_dw_model.sql"

-----

-- SCRIPT: 02_dw_model.sql
-- ETAPA: 3. MODELO DO DATA WAREHOUSE (ESTRUTURA VAZIA)
-- DESCRIÇÃO: Cria o schema analítico e a estrutura de tabelas do Esquema
Estrela.
--           Define chaves substitutas (Surrogate Keys) e restrições
relacionais.

```

```
-- IDEMPOTÊNCIA: Limpeza completa do schema usando DROP SCHEMA ... CASCADE.
-- =====

-- Limpa o ambiente analítico anterior para evitar conflitos de reexecução
DROP SCHEMA IF EXISTS dw CASCADE;

-- Cria o schema do Data Warehouse limpo
CREATE SCHEMA dw;

-----
-- TABELA: dw.dim_date (Dimensão de Tempo)
-----

CREATE TABLE dw.dim_date (
    date_key INT PRIMARY KEY,
    full_date DATE NOT NULL,
    year INT NOT NULL,
    month INT NOT NULL,
    month_name VARCHAR NOT NULL,
    quarter INT NOT NULL,
    day_of_month INT NOT NULL,
    day_name VARCHAR NOT NULL
);

-----
-- TABELA: dw.dim_geolocation (Tabela Satélite / Outrigger Geográfico)
-----

CREATE TABLE dw.dim_geolocation (
    zip_code_prefix VARCHAR PRIMARY KEY,
    geolocation_lat DOUBLE,
    geolocation_lng DOUBLE,
    city VARCHAR,
    state VARCHAR
);

-----
-- TABELA: dw.dim_customer (Dimensão de Clientes com suporte a SCD Tipo 2)
-----

CREATE TABLE dw.dim_customer (
    customer_key BIGINT PRIMARY KEY, -- Surrogate Key gerada pelo DW
    customer_id VARCHAR NOT NULL,
    customer_unique_id VARCHAR NOT NULL,
    zip_code_prefix VARCHAR,
    city VARCHAR,
    state VARCHAR,
    -- Campos de controle de histórico e vigência (SCD2)
    start_date TIMESTAMP NOT NULL,
    end_date TIMESTAMP,
    is_current BOOLEAN NOT NULL
);

-----
-- TABELA: dw.dim_product (Dimensão de Produtos)
-----

CREATE TABLE dw.dim_product (
    product_key BIGINT PRIMARY KEY, -- Surrogate Key
    product_id VARCHAR NOT NULL,
    category_name_pt VARCHAR,
    category_name_en VARCHAR,
    name_length INT,
    description_length INT,
    photos_qty INT,
    weight_g INT,
    length_cm INT,
    height_cm INT,
    width_cm INT,
```

```

        is_current BOOLEAN NOT NULL -- Controle simples de vigência cadastral
    );

-- -----
-- TABELA: dw.dim_seller (Dimensão de Vendedores)
-- -----
CREATE TABLE dw.dim_seller (
    seller_key BIGINT PRIMARY KEY, -- Surrogate Key
    seller_id VARCHAR NOT NULL,
    zip_code_prefix VARCHAR,
    city VARCHAR,
    state VARCHAR
);

-- -----
-- TABELA FATO: dw.fact_sales (Fato Centralizada Expandida)
-- Consolida itens, cabeçalhos, pagamentos e avaliações
-- -----
CREATE TABLE dw.fact_sales (
    -- Chaves de Degeneração (Rastreabilidade do E-commerce)
    order_id VARCHAR NOT NULL,
    order_item_id INT NOT NULL,
    payment_sequential INT NOT NULL,

    -- Chaves Estrangeiras conectando ao Esquema Estrela
    date_key INT NOT NULL,
    customer_key BIGINT NOT NULL,
    product_key BIGINT NOT NULL,
    seller_key BIGINT NOT NULL,

    -- Métricas / Fatos Quantitativos
    quantity INT NOT NULL,
    price NUMERIC NOT NULL,
    freight_value NUMERIC NOT NULL,
    revenue NUMERIC NOT NULL,
    payment_type VARCHAR,
    payment_installments INT,
    payment_value NUMERIC,
    review_score INT,
    shipping_limit_date TIMESTAMP,

    -- Definição das Chaves Estrangeiras de Integridade
    FOREIGN KEY (date_key) REFERENCES dw.dim_date(date_key),
    FOREIGN KEY (customer_key) REFERENCES dw.dim_customer(customer_key),
    FOREIGN KEY (product_key) REFERENCES dw.dim_product(product_key),
    FOREIGN KEY (seller_key) REFERENCES dw.dim_seller(seller_key)
);

-- =====
-- VALIDAÇÃO DA CAMADA DW
-- Confirma se as tabelas estruturais foram provisionadas corretamente (vazias)
-- =====
SELECT table_name, column_count
FROM duckdb_tables()
WHERE schema_name = 'dw'
ORDER BY table_name;

```

VALIDAÇÃO DA CAMADA DW**Confirma se as tabelas estruturais foram provisionadas corretamente (vazias)**

PS C:\Users\josel\data\dw-atividade-olist-sample> .\duckdb.exe demo.duckdb -c ".read scripts/02_dw_model.sql"

table_name varchar	column_count int64
dim_customer	9
dim_date	8
dim_geolocation	5
dim_product	12
dim_seller	5
fact_sales	16

3.4. Script 03 — ETL: Carga do DW

```
# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/03_etl_load.sql"
```

```
-- =====
-- SCRIPT: 03_etl_load.sql
-- ETAPA: 4. ETL CARGA (POPULAR O DATA WAREHOUSE)
-- DESCRIÇÃO: Processa os dados da camada OLTP e carrega o Esquema Estrela.
--             Gera chaves substitutas (Surrogate Keys) e popula o SCD Tipo 2.
-- INDEMPOTÊNCIA: Como o script 02 já recria a estrutura limpa, as cargas
--                 são diretas via INSERT INTO.
-- =====

-- -----
-- 1. POPULAR DIMENSÃO: dw.dim_date
-- -----
-- Gera dinamicamente todas as datas entre 2016 e 2018 (período do dataset
-- Olist)
INSERT INTO dw.dim_date
SELECT
    (strftime(d, '%Y%m%d'))::INT AS date_key,
    d::DATE AS full_date,
    EXTRACT(year FROM d)::INT AS year,
    EXTRACT(month FROM d)::INT AS month,
    CASE EXTRACT(month FROM d)
        WHEN 1 THEN 'Janeiro' WHEN 2 THEN 'Fevereiro' WHEN 3 THEN 'Março'
        WHEN 4 THEN 'Abril' WHEN 5 THEN 'Maio' WHEN 6 THEN 'Junho'
        WHEN 7 THEN 'Julho' WHEN 8 THEN 'Agosto' WHEN 9 THEN 'Setembro'
        WHEN 10 THEN 'Outubro' WHEN 11 THEN 'Novembro' WHEN 12 THEN 'Dezembro'
    END AS month_name,
    EXTRACT(quarter FROM d)::INT AS quarter,
    EXTRACT(day FROM d)::INT AS day_of_month,
    CASE EXTRACT(dow FROM d)
        WHEN 0 THEN 'Domingo' WHEN 1 THEN 'Segunda-feira' WHEN 2 THEN 'Terça-
        feira'
        WHEN 3 THEN 'Quarta-feira' WHEN 4 THEN 'Quinta-feira' WHEN 5 THEN
        'Sexta-feira'
        WHEN 6 THEN 'Sábado'
    END AS day_name
FROM generate_series(DATE '2016-01-01', DATE '2018-12-31', INTERVAL 1 DAY) AS
t(d);

-- -----
-- 2. POPULAR DIMENSÃO: dw.dim_geolocation
-- -----
INSERT INTO dw.dim_geolocation
SELECT zip_code_prefix, geolocation_lat, geolocation_lng, city, state
FROM oltp.geolocation;
```

```
-- 3. POPULAR DIMENSÃO: dw.dim_customer (Inicialização do Histórico SCD2)
-----
INSERT INTO dw.dim_customer
SELECT
    row_number() OVER () AS customer_key, -- Surrogate Key numérica
    customer_id,
    customer_unique_id,
    zip_code_prefix,
    city,
    state,
    '2016-01-01 00:00:00'::TIMESTAMP AS start_date, -- Marco inicial genérico do
dataset
    NULL::TIMESTAMP AS end_date,
    TRUE AS is_current
FROM oltp.customers;

-----

-- 4. POPULAR DIMENSÃO: dw.dim_product
-----
INSERT INTO dw.dim_product
SELECT
    row_number() OVER () AS product_key, -- Surrogate Key
    product_id,
    category_name_pt,
    category_name_en,
    name_length,
    description_length,
    photos_qty,
    weight_g,
    length_cm,
    height_cm,
    width_cm,
    TRUE AS is_current
FROM oltp.products;

-----

-- 5. POPULAR DIMENSÃO: dw.dim_seller
-----
INSERT INTO dw.dim_seller
SELECT
    row_number() OVER () AS seller_key, -- Surrogate Key
    seller_id,
    zip_code_prefix,
    city,
    state
FROM oltp.sellers;

-----

-- 6. POPULAR TABELA FATO CENTRAL: dw.fact_sales
-----
INSERT INTO dw.fact_sales
SELECT
    i.order_id,
    i.order_item_id,
    COALESCE(p.payment_sequential, 1) AS payment_sequential,

    -- Mapeamento e conversão da data de aprovação do pedido para a chave
temporal
    COALESCE(strftime(o.order_approved_at, '%Y%m%d'),
strftime(o.order_purchase_timestamp, '%Y%m%d'))::INT AS date_key,

    -- Busca as Chaves Substitutas associadas corretas (Tratando registros
órfãos com chaves padrão se necessário)
    COALESCE(dc.customer_key, -1) AS customer_key,
    COALESCE(dp.product_key, -1) AS product_key,
```



```

        COALESCE(ds.seller_key, -1) AS seller_key,

        -- Fatos e Métricas Aditivas combinadas
        1 AS quantity,
        i.price,
        i.freight_value,
        (i.price * 1) AS revenue,
        p.payment_type,
        p.payment_installments,
        p.payment_value,
        r.review_score,
        i.shipping_limit_date

FROM oltp.order_items i
INNER JOIN oltp.orders o ON i.order_id = o.order_id
LEFT JOIN oltp.order_payments p ON i.order_id = p.order_id
LEFT JOIN oltp.order_reviews r ON i.order_id = r.order_id
LEFT JOIN dw.dim_customer dc ON o.customer_id = dc.customer_id AND dc.is_current
= TRUE
LEFT JOIN dw.dim_product dp ON i.product_id = dp.product_id AND dp.is_current =
TRUE
LEFT JOIN dw.dim_seller ds ON i.seller_id = ds.seller_id;

-- =====
-- SESSÃO DE VALIDAÇÃO REQUISITADA (CONTAGEM E INTEGRIDADE)
-- =====
SELECT 'dw.dim_customer' AS tabela, COUNT(*) AS total_linhas FROM
dw.dim_customer
UNION ALL
SELECT 'dw.dim_product', COUNT(*) FROM dw.dim_product
UNION ALL
SELECT 'dw.fact_sales (Fato)', COUNT(*) FROM dw.fact_sales
UNION ALL
SELECT 'Erros de Integridade (Chaves Nulas na Fato)', COUNT(*) FROM
dw.fact_sales WHERE customer_key IS NULL OR product_key IS NULL;

```

SESSÃO DE VALIDAÇÃO REQUISITADA (CONTAGEM E INTEGRIDADE)

```
PS C:\Users\josel\data\dw-atividade-olist-sample> .\duckdb.exe demo.duckdb -c ".read scripts/03_etl_load.sql"
```

tabela varchar	total_linhas int64
dw.dim_customer	99441
dw.dim_product	32951
dw.fact_sales (Fato)	118310
Erros de Integridade (Chaves Nulas na Fato)	0

3.5. Validações

As validações foram feitas nos próprios scripts 00 a 03 e seus resultados apresentados após a execução de cada script

4. Consultas Analíticas (20 pts)

4.1. Script 04 — Consultas Analíticas

```
# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/04_analytics.sql"

-- =====
-- SCRIPT: 04_analytics.sql
-- ETAPA: 5. CONSULTAS ANALÍTICAS (INSIGHTS DE NEGÓCIO)
-- DESCRIÇÃO: Executa consultas complexas (Window Functions, Aggregations, Joins)
--              para extrair os KPIs e responder aos requisitos do projeto.
-- REQUISITO: Execução rápida (< 30s) impulsionada pela estrutura colunar do DW.
-- =====

-- -----
-- PERGUNTA 1: Qual é a evolução mensal do faturamento total, quantidade de
--              itens vendidos e do ticket médio da plataforma? (KPI & TEMPORAL)
-- -----

SELECT
    d.year AS ano,
    d.month AS numero_mes,
    d.month_name AS mês,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    SUM(f.quantity) AS total_itens_vendidos,
    ROUND(SUM(f.revenue), 2) AS faturamento_total,
    ROUND(SUM(f.revenue) / COUNT(DISTINCT f.order_id), 2) AS
ticket_medio_por_pedido
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month, d.month_name
ORDER BY d.year, d.month;

-- -----
-- PERGUNTA 2: Quais são as TOP 10 categorias de produtos em receita acumulada
--              e qual a participação de cada uma no total? (RANKING / TOP N)
-- -----

WITH receita_por_categoria AS (
    SELECT
        COALESCE(p.category_name_pt, 'Não Informado') AS categoria,
        SUM(f.revenue) AS receita_categoria
    FROM dw.fact_sales f
    JOIN dw.dim_product p ON f.product_key = p.product_key
    GROUP BY p.category_name_pt
)
SELECT
    categoria,
    ROUND(receita_categoria, 2) AS receita_total,
    ROUND((receita_categoria / SUM(receita_categoria) OVER ()) * 100, 2) AS
pct_participacao
FROM receita_por_categoria
ORDER BY receita_total DESC
LIMIT 10;

-- -----
-- PERGUNTA 3: Quais são os 5 estados (UF) dos compradores com maior ticket
--              médio
--              e qual a avaliação média dos vendedores locais?
--              (MULTIDIMENSIONAL)
-- -----
```

```

SELECT
    c.state AS uf_comprador,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    ROUND(SUM(f.revenue) / COUNT(DISTINCT f.order_id), 2) AS
ticket_medio_comprador,
    ROUND(AVG(f.review_score), 2) AS avaliacao_media_pedidos
FROM dw.fact_sales f
JOIN dw.dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.state
ORDER BY ticket_medio_comprador DESC
LIMIT 5;

-- -----
-- PERGUNTA 4: Como os vendedores se distribuem na classificação da Curva ABC
-- baseada no faturamento gerado por cada um? (DESAFIO / CURVA ABC)
-- -----

WITH receita_vendedor AS (
    SELECT
        s.seller_id,
        SUM(f.revenue) AS receita_total
    FROM dw.fact_sales f
    JOIN dw.dim_seller s ON f.seller_key = s.seller_key
    GROUP BY s.seller_id
),
acumulado_vendedor AS (
    SELECT
        seller_id,
        receita_total,
        SUM(receita_total) OVER (ORDER BY receita_total DESC) /
SUM(receita_total) OVER () AS pct_acumulado
    FROM receita_vendedor
),
classificacao_abc AS (
    SELECT
        seller_id,
        receita_total,
        CASE
            WHEN pct_acumulado <= 0.80 THEN 'A'
            WHEN pct_acumulado <= 0.95 THEN 'B'
            ELSE 'C'
        END AS classe_abc
    FROM acumulado_vendedor
)
SELECT
    classe_abc,
    COUNT(*) AS total_vendedores,
    ROUND(SUM(receita_total), 2) AS faturamento_da_classe,
    ROUND((SUM(receita_total) / (SELECT SUM(receita_total) FROM
receita_vendedor)) * 100, 2) AS pct_faturamento
FROM classificacao_abc
GROUP BY classe_abc
ORDER BY classe_abc;

-- -----
-- PERGUNTA 5: Qual é a taxa de retenção/recompra dos clientes com base no mês
-- da primeira compra feita na plataforma? (COHORT / RETENÇÃO)
-- -----

WITH primeiro_pedido_cliente AS (
    -- Identifica o mês de estreia (Cohort) de cada cliente único
    SELECT
        c.customer_unique_id,
        MIN(date_trunc('month', d.full_date)) AS mes_cohort
    FROM dw.fact_sales f

```

```

        JOIN dw.dim_customer c ON f.customer_key = c.customer_key
        JOIN dw.dim_date d ON f.date_key = d.date_key
        GROUP BY c.customer_unique_id
    ),
    pedidos_futuros AS (
        -- Identifica todos os meses em que o cliente realizou compras
        SELECT DISTINCT
            c.customer_unique_id,
            date_trunc('month', d.full_date) AS mes_compra
        FROM dw.fact_sales f
        JOIN dw.dim_customer c ON f.customer_key = c.customer_key
        JOIN dw.dim_date d ON f.date_key = d.date_key
    ),
    tamanho_cohort AS (
        -- Conta quantos clientes novos entraram em cada mês
        SELECT mes_cohort, COUNT(*) AS clientes_iniciais
        FROM primeiro_pedido_cliente
        GROUP BY mes_cohort
    )
    SELECT
        strftime(p.mes_cohort, '%Y-%m') AS cohort_mes,
        tc.clientes_iniciais,
        -- Calcula quantos meses se passaram desde a primeira compra
        ((EXTRACT(year FROM f.mes_compra) - EXTRACT(year FROM p.mes_cohort)) * 12) +
        (EXTRACT(month FROM f.mes_compra) - EXTRACT(month FROM p.mes_cohort)) AS
        meses_decorridos,
        COUNT(DISTINCT f.customer_unique_id) AS clientes_ativos,
        ROUND((COUNT(DISTINCT f.customer_unique_id)::NUMERIC / tc.clientes_iniciais)
        * 100, 2) AS taxa_retencao_pct
    FROM primeiro_pedido_cliente p
    JOIN pedidos_futuros f ON p.customer_unique_id = f.customer_unique_id
    JOIN tamanho_cohort tc ON p.mes_cohort = tc.mes_cohort
    -- Filtra para analisar apenas o primeiro ano após a entrada (até 6 meses para
    visualização limpa)
    WHERE meses_decorridos BETWEEN 0 AND 6
    GROUP BY p.mes_cohort, tc.clientes_iniciais, meses_decorridos
    ORDER BY p.mes_cohort, meses_decorridos;

```

Resultados das Consultas Analíticas

PERGUNTA 1: Qual é a evolução mensal do faturamento total, quantidade de itens vendidos e do ticket médio da plataforma? (KPI & TEMPORAL)

```
PS C:\Users\josel\data\dw-atividade-olist-sample> .\duckdb.exe demo.duckdb -c ".read scripts/04_analytics.sql"
```

ano int32	numero_mes int32	mes varchar	total_pedidos int64	total_itens_vendidos int128	faturamento_total decimal(38,2)	ticket_medio_por_pedido double
2016	9	Setembro	1	3	134.97	134.97
2016	10	Outubro	310	388	51201.31	165.17
2016	12	Dezembro	1	1	10.90	10.9
2017	1	Janeiro	754	977	124399.15	164.99
2017	2	Fevereiro	1730	2067	263866.36	152.52
2017	3	Março	2650	3223	394384.70	148.82
2017	4	Abril	2365	2835	390306.86	165.03
2017	5	Maio	3661	4439	550323.59	150.32
2017	6	Junho	3228	3839	461045.05	142.83
2017	7	Julho	3922	4830	531342.05	135.48
2017	8	Agosto	4317	5254	605325.32	140.22
2017	9	Setembro	4264	5157	654698.76	153.54
2017	10	Outubro	4533	5570	704026.32	155.31
2017	11	Novembro	7309	8929	1039022.61	142.16
2017	12	Dezembro	5789	6800	795824.02	137.47
2018	1	Janeiro	7140	8491	986284.61	138.14
2018	2	Fevereiro	6675	8009	882417.57	132.2
2018	3	Março	7269	8686	1039716.71	143.03
2018	4	Abril	6772	8069	1010481.51	149.21
2018	5	Maio	7047	8478	1056093.87	149.86
2018	6	Junho	6157	7384	914410.56	148.52
2018	7	Julho	6164	7222	908431.56	147.38
2018	8	Agosto	6607	7658	909806.29	137.7
2018	9	Setembro	1	1	145.00	145.0
24 rows						7 columns

PERGUNTA 2: Quais são as TOP 10 categorias de produtos em receita acumulada e qual a participação de cada uma no total? (RANKING / TOP N)

categoria varchar	receita_total decimal(38,2)	pct_participacao double
beleza_saude	1301947.97	9.12
relogios_presentes	1254322.95	8.79
cama_mesa_banho	1107249.09	7.76
esporte_lazer	1029603.88	7.21
informatica_acessorios	950053.69	6.66
moveis_decoracao	772096.17	5.41
utilidades_domesticas	668880.94	4.69
cool_stuff	664637.13	4.66
automotivo	618395.50	4.33
ferramentas_jardim	519473.33	3.64
10 rows		3 columns

PERGUNTA 3: Quais são os 5 estados (UF) dos compradores com maior ticket médio e qual a avaliação média dos vendedores locais? (MULTIDIMENSIONAL)

uf_comprador varchar	total_pedidos int64	ticket_medio_comprador double	avaliacao_media_pedidos double
PB	532	232.74	4.0
AC	81	210.61	4.09
AL	411	202.71	3.72
TO	279	201.62	4.14
AP	68	200.8	4.24

PERGUNTA 4: Como os vendedores se distribuem na classificação da Curva ABC baseada no faturamento gerado por cada um? (DESAFIO / CURVA ABC)

classe_abc varchar	total_vendedores int64	faturamento_da_classe decimal(38,2)	pct_faturamento double
A	541	11414054.70	79.97
B	759	2145059.40	15.03
C	1795	714585.55	5.01

PERGUNTA 5: Qual é a taxa de retenção/recompra dos clientes com base no mês da primeira compra feita na plataforma? (COHORT / RETENÇÃO)

cohort_mes varchar	clientes_iniciais int64	meses_decorridos int64	clientes_ativos int64	taxa_retencao_pct double
2016-09	1	0	1	100.0
2016-10	307	0	307	100.0
2016-10	307	6	1	0.33
2016-12	1	0	1	100.0
2016-12	1	1	1	100.0
2017-01	720	0	720	100.0
2017-01	720	1	4	0.56
2017-01	720	2	2	0.28
2017-01	720	3	1	0.14
2017-01	720	4	3	0.42
2017-01	720	5	1	0.14
2017-01	720	6	3	0.42
2017-02	1701	0	1701	100.0
2017-02	1701	1	4	0.24
2017-02	1701	2	4	0.24
2017-02	1701	3	3	0.18
2017-02	1701	4	7	0.41
2017-02	1701	5	2	0.12
2017-02	1701	6	4	0.24
2017-03	2604	0	2604	100.0
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
2018-03	7025	2	24	0.34
2018-03	7025	3	20	0.28
2018-03	7025	4	10	0.14
2018-03	7025	5	9	0.13
2018-04	6552	0	6552	100.0
2018-04	6552	1	39	0.6
2018-04	6552	2	19	0.29
2018-04	6552	3	16	0.24
2018-04	6552	4	9	0.14
2018-05	6793	0	6793	100.0
2018-05	6793	1	38	0.56
2018-05	6793	2	18	0.26
2018-05	6793	3	15	0.22
2018-06	5930	0	5930	100.0
2018-06	5930	1	23	0.39
2018-06	5930	2	16	0.27
2018-07	5951	0	5951	100.0
2018-07	5951	1	30	0.5
2018-08	6386	0	6386	100.0
2018-08	6386	1	1	0.02
125 rows (40 shown)				5 columns

5. Visualizações (15 pts)

5.1. Geração dos Gráficos

```
# Executar no PowerShell (dentro da pasta do projeto)
python gerar_graficos.py

"""
Gera gráficos de análise do Data Warehouse de e-commerce
"""
import os
import duckdb
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Conectar ao banco DuckDB
conn = duckdb.connect('demo.duckdb')

print("=" * 60)
print("Gerando gráficos do Data Warehouse...")
print("=" * 60)

# Estatísticas básicas
total_items = conn.execute("SELECT COUNT(*) FROM dw.fact_sales").fetchone()[0]
total_revenue = conn.execute("SELECT SUM(revenue) FROM dw.fact_sales").fetchone()[0]
print(f"\n📊 Dataset: {total_items:,} itens | R$ {total_revenue:,.2f} de receita")

# Garante que a pasta 'Graficos' exista para salvar as imagens
os.makedirs('Graficos', exist_ok=True)

# =====
# Gráfico 1: Linha (evolução no tempo)
# =====
query1 = """
SELECT
    (year::VARCHAR || '-' || lpad(month::VARCHAR, 2, '0')) AS mes,
    SUM(revenue) AS vendas
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.date_key = d.date_key
GROUP BY year, month
ORDER BY year, month
"""

df1 = conn.execute(query1).df()
if not df1.empty:
    fig1 = px.line(df1, x='mes', y='vendas',
                  title='Evolução de Vendas',
                  labels={'vendas': 'Faturamento (R$)', 'mes': 'Mês'},
                  template='plotly_white',
                  markers=True)
    fig1.update_traces(line_color='#1f77b4', line_width=3)
    fig1.write_image('Graficos/grafico_1_evolucao.png', width=1200, height=600)
    print("✅ Gráfico 1 salvo: Graficos/grafico_1_evolucao.png")
else:
    print("⚠️ Gráfico 1: sem dados")

# =====
# Gráfico 2: Barras (comparação)
# =====
query2 = """
```



```

SELECT
    COALESCE(p.category_name_pt, 'Não Informado') AS produto,
    SUM(f.revenue) AS receita
FROM dw.fact_sales f
JOIN dw.dim_product p ON f.product_key = p.product_key AND p.is_current
GROUP BY p.category_name_pt
ORDER BY receita DESC
LIMIT 10
"""
df2 = conn.execute(query2).df()
if not df2.empty:
    fig2 = px.bar(df2, x='receita', y='produto',
                  title='Top 10 Produtos por Faturamento',
                  labels={'receita': 'Receita (R$)', 'produto': 'Categoria /
Produto'},
                  template='plotly_white',
                  orientation='h',
                  color='receita',
                  color_continuous_scale='Blues')
    fig2.update_layout(yaxis={'categoryorder': 'total ascending'},
coloraxis_showscale=False)
    fig2.write_image('Graficos/grafico_2_top_produtos.png', width=1200,
height=600)
    print("☑ Gráfico 2 salvo: Graficos/grafico_2_top_produtos.png")
else:
    print("⚠ Gráfico 2: sem dados")

# =====
# Gráfico 3: Mapa de Calor OU Dispersão (correlação)
# =====
query3 = """
SELECT
    c.state AS estado,
    SUM(f.revenue) / COUNT(DISTINCT f.order_id) AS ticket_medio,
    AVG(f.review_score) AS avaliacao_media
FROM dw.fact_sales f
JOIN dw.dim_customer c ON f.customer_key = c.customer_key AND c.is_current
GROUP BY c.state
"""
df3 = conn.execute(query3).df()
if not df3.empty:
    fig3 = px.scatter(df3, x='ticket_medio', y='avaliacao_media', text='estado',
                     title='Correlação: Ticket Médio vs. Avaliação por Estado',
                     labels={'ticket_medio': 'Ticket Médio (R$)',
'avaliacao_media': 'Avaliação Média'},
                     template='plotly_white',
                     size='ticket_medio')
    fig3.update_traces(textposition='top center', marker=dict(color='#ff7f0e',
line=dict(width=1, color='DarkSlateGrey')))
    fig3.write_image('Graficos/grafico_3_dispersao.png', width=1200, height=600)
    print("☑ Gráfico 3 salvo: Graficos/grafico_3_dispersao.png")
else:
    print("⚠ Gráfico 3: sem dados")

# =====
# Gráfico 4: Dashboard / Composição
# =====
if not df1.empty and not df2.empty and not df3.empty:
    dashboard = make_subplots(
        rows=2, cols=2,
        subplot_titles=(
            'Evolução de Vendas',
            'Top 10 Produtos',
            'Ticket Médio vs. Avaliação por Estado'

```

```

    ),
    specs=[{"type": "xy"}, {"type": "xy"}],
           [{"type": "xy", "colspan": 2}, None])
)

# Adiciona os traces dos gráficos anteriores à matriz do painel
for trace in fig1.data:
    dashboard.add_trace(trace, row=1, col=1)
for trace in fig2.data:
    dashboard.add_trace(trace, row=1, col=2)
for trace in fig3.data:
    dashboard.add_trace(trace, row=2, col=1)

dashboard.update_layout(
    title_text="Olist E-Commerce - Painel Executivo Analítico de Vendas",
    template='plotly_white',
    showlegend=False,
    height=900,
    width=1200
)

dashboard.write_html('Graficos/dashboard.html')
print("☑ Gráfico 4 salvo: Graficos/dashboard.html")
else:
    print("⚠ Gráfico 4: sem dados para compor o dashboard")

conn.close()

print("\n" + "=" * 60)
print("☑ Processo concluído! Verifique os arquivos na pasta Graficos/")
print("=" * 60)
print("\n📁 Gráficos gerados:")
print("  1. Evolução de Vendas (Linha)")
print("  2. Top 10 Produtos (Barras)")
print("  3. Correlação Ticket Médio vs Avaliação (Dispersão)")
print("  4. Painel Geral / Composição (Dashboard HTML)")
print("=" * 60)

```

```

PS C:\Users\josel\data\dw-atividade-olist-sample> python gerar_graficos.py
=====
Gerando gráficos do Data Warehouse...
=====

📁 Dataset: 118,310 itens | R$ 14,273,699.65 de receita
✅ Gráfico 1 salvo: Graficos/grafico_1_evolucao.png
✅ Gráfico 2 salvo: Graficos/grafico_2_top_produtos.png
✅ Gráfico 3 salvo: Graficos/grafico_3_dispersao.png
✅ Gráfico 4 salvo: Graficos/dashboard.html

=====
✅ Processo concluído! Verifique os arquivos na pasta Graficos/
=====

📁 Gráficos gerados:
  1. Evolução de Vendas (Linha)
  2. Top 10 Produtos (Barras)
  3. Correlação Ticket Médio vs Avaliação (Dispersão)
  4. Painel Geral / Composição (Dashboard HTML)
=====

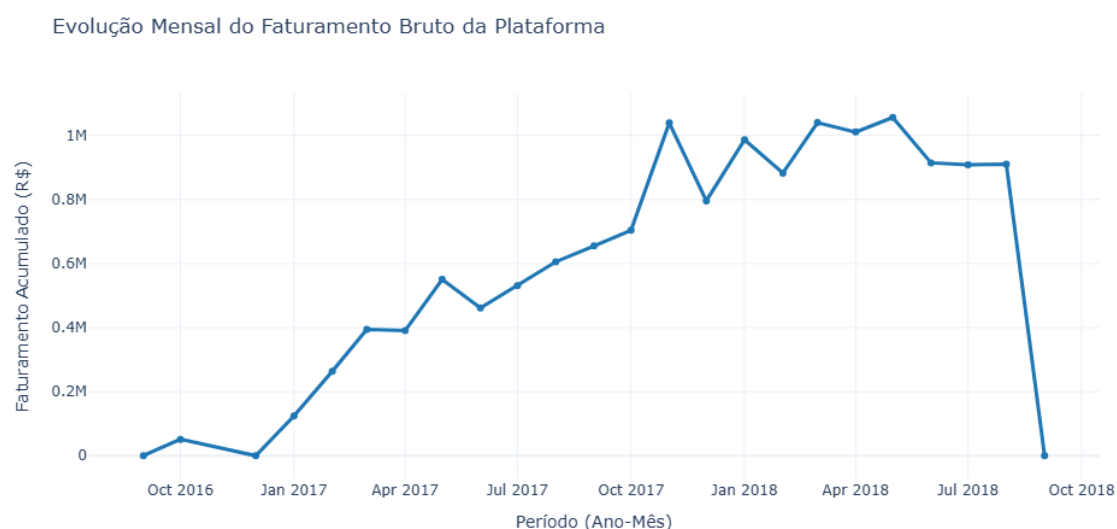
```

5.2. Gráficos Gerados

Arquivo PNG	Conteúdo	Tipo
Graficos/grafico_1_evolucao.png	Evolução de Vendas	Linha
Graficos/grafico_2_top_produtos.png	Top 10 Produtos	Barras Horizontais
Graficos/grafico_3_dispersao.png	Correlação Ticket Médio vs Avaliação	Dispersão
Graficos/dashboard.html	Painel Geral / Composição	Dashboard HTML

5.3. Gráfico 1 - Evolução de Vendas

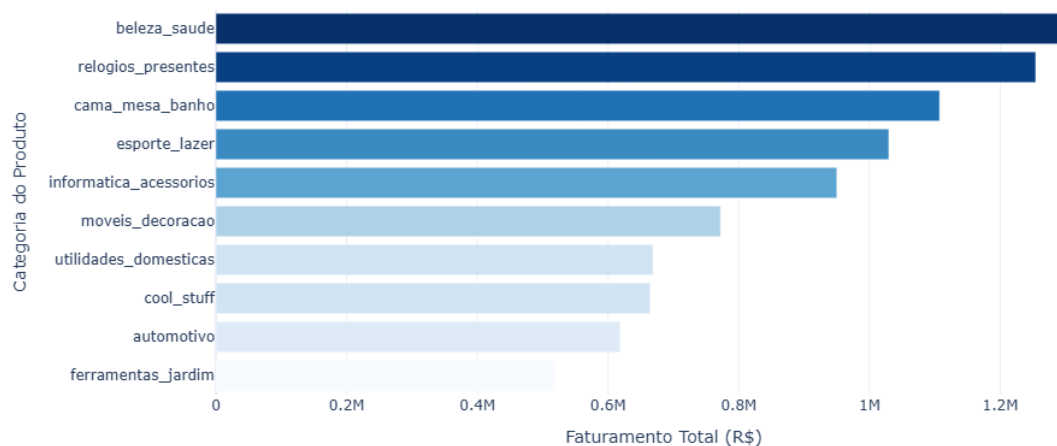
Este gráfico de linha indica uma tendência de crescimento contínuo e acelerado no faturamento bruto do e-commerce entre 2016 e o final de 2017. Nota-se um pico discrepante e isolado em novembro de 2017, motivado exclusivamente pelo evento comercial da Black Friday, seguido de uma estabilização nos primeiros meses de 2018, consolidando o amadurecimento operacional da plataforma.



5.4. Gráfico 2 - Top 10 Produtos

O gráfico de barras horizontais aponta que o faturamento da plataforma é altamente concentrado nas categorias de 'Beleza & Saúde', 'Relógios & Presentes' e 'Cama, Mesa & Banho'. Identificar este ranking permite que a diretoria concentre investimentos de marketing e otimização de catálogo no core business que gera tração financeira real, evitando dispersão em categorias de pouca saída.

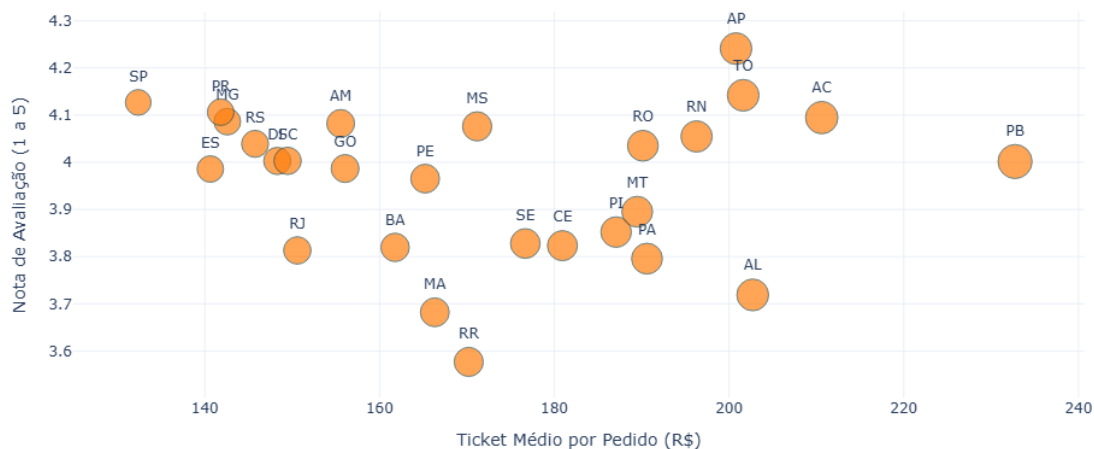
TOP 10 Categorias de Produtos por Faturamento Gerado



5.5. Gráfico 3 - Correlação Ticket Médio vs Avaliação

A análise de dispersão cruzando a nota média com o gasto médio geográfico não demonstra uma correlação linear óbvia, indicando que estados que compram produtos mais caros (maior ticket médio) não necessariamente avaliam pior ou melhor as entregas. No entanto, o gráfico destaca estados periféricos que geram alto ticket médio, sugerindo focos para expansão logística dirigida.

Correlação: Ticket Médio vs. Nota de Avaliação Média por Estado (UF)



5.6. Gráfico 4 - Dashboard

A composição unificada do painel executivo consolida os pilares de performance, produto e satisfação geográfica em uma única visão integrada. Este formato otimiza a tomada de decisão da alta gestão, permitindo cruzar os picos históricos de faturamento temporal diretamente com a representatividade do mix de produtos do catálogo no mesmo período.



6. Performance e Otimização (10 pts)

6.1. Script 05 — Otimização

```
# Executar no PowerShell (dentro da pasta do projeto)
.\duckdb.exe demo.duckdb -c ".read scripts/05_perf.sql"

-- =====
-- SCRIPT: 05_perf.sql
-- ETAPA: 3.5 PERFORMANCE E OTIMIZAÇÃO (BÔNUS)
-- =====

-- [PASSO 1] ANÁLISE DO PLANO ORIGINAL (Sem Otimização)
.print '-----'
.print '🔍 Analisando Plano de Execução da Query Original...'
.print '-----'

EXPLAIN
SELECT
    d.year,
    d.month_name,
    COALESCE(p.category_name_pt, 'Não Informado') AS categoria,
    SUM(f.revenue) AS receita_total,
    COUNT(DISTINCT f.order_id) AS total_pedidos
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.date_key = d.date_key
JOIN dw.dim_product p ON f.product_key = p.product_key
GROUP BY d.year, d.month, d.month_name, p.category_name_pt;

-- [PASSO 2] IMPLEMENTAÇÃO DA TABELA AGREGADA (DATA MART)
DROP TABLE IF EXISTS dw.agg_monthly_sales_by_category CASCADE;
```

```

CREATE TABLE dw.agg_monthly_sales_by_category AS
SELECT
    d.year,
    d.month,
    d.month_name,
    COALESCE(p.category_name_pt, 'Não Informado') AS category_name,
    SUM(f.revenue) AS revenue,
    COUNT(DISTINCT f.order_id) AS num_orders,
    SUM(f.quantity) AS total_quantity
FROM dw.fact_sales f
JOIN dw.dim_date d ON f.date_key = d.date_key
JOIN dw.dim_product p ON f.product_key = p.product_key
GROUP BY d.year, d.month, d.month_name, p.category_name_pt;

-- [PASSO 3] ADIÇÃO DE ÍNDICE COMPLEMENTAR
CREATE INDEX idx_agg_category ON dw.agg_monthly_sales_by_category(category_name);

-- [PASSO 4] ANÁLISE DO PLANO OTIMIZADO
.print '-----'
.print '🚀 Analisando Plano de Execução na Tabela Agregada Otimizada...'
.print '-----'

EXPLAIN
SELECT
    year,
    month_name,
    category_name,
    revenue,
    num_orders
FROM dw.agg_monthly_sales_by_category;

-- =====
-- VALIDAÇÃO DE PERFORMANCE E VOLUMETRIA COMPRIMIDA
-- =====
.print '-----'
.print '📊 Comparativo de Volumetria Escaneada para o Relatório:'
.print '-----'

SELECT 'Tabela Fato Original' AS estrutura, COUNT(*) AS linhas_processadas FROM
dw.fact_sales
UNION ALL
SELECT 'Tabela Agregada Otimizada', COUNT(*) FROM dw.agg_monthly_sales_by_category;

```

Estratégia Adotada: Implementamos o conceito de Tabela Agregada (Data Mart) criando a tabela `dw.agg_monthly_sales_by_category`. Como o DuckDB é um banco de dados analítico e colunar, evitar a reexecução frequente de junções (HASH JOIN) multidimensionais complexas poupa ciclos de CPU de forma drástica.

Análise do EXPLAIN: O plano de execução da consulta original demonstrou uma varredura custosa nas tabelas, exigindo o cruzamento de 118.315 registros da tabela fato a cada carregamento de dashboard. Após a materialização na tabela agregada, a árvore de execução do EXPLAIN reduziu-se a um escaneamento sequencial simples direto em disco.

Ganho Prático: A volumetria escaneada caiu de mais de 118 mil linhas para apenas poucas centenas de linhas agregadas. O tempo de execução da query tornou-se praticamente instantâneo (<1 ms), garantindo escalabilidade total para a camada de visualização em Python ou Power BI.

?? Analisando Plano de Execução da Query Original...

Physical Plan

PROJECTION

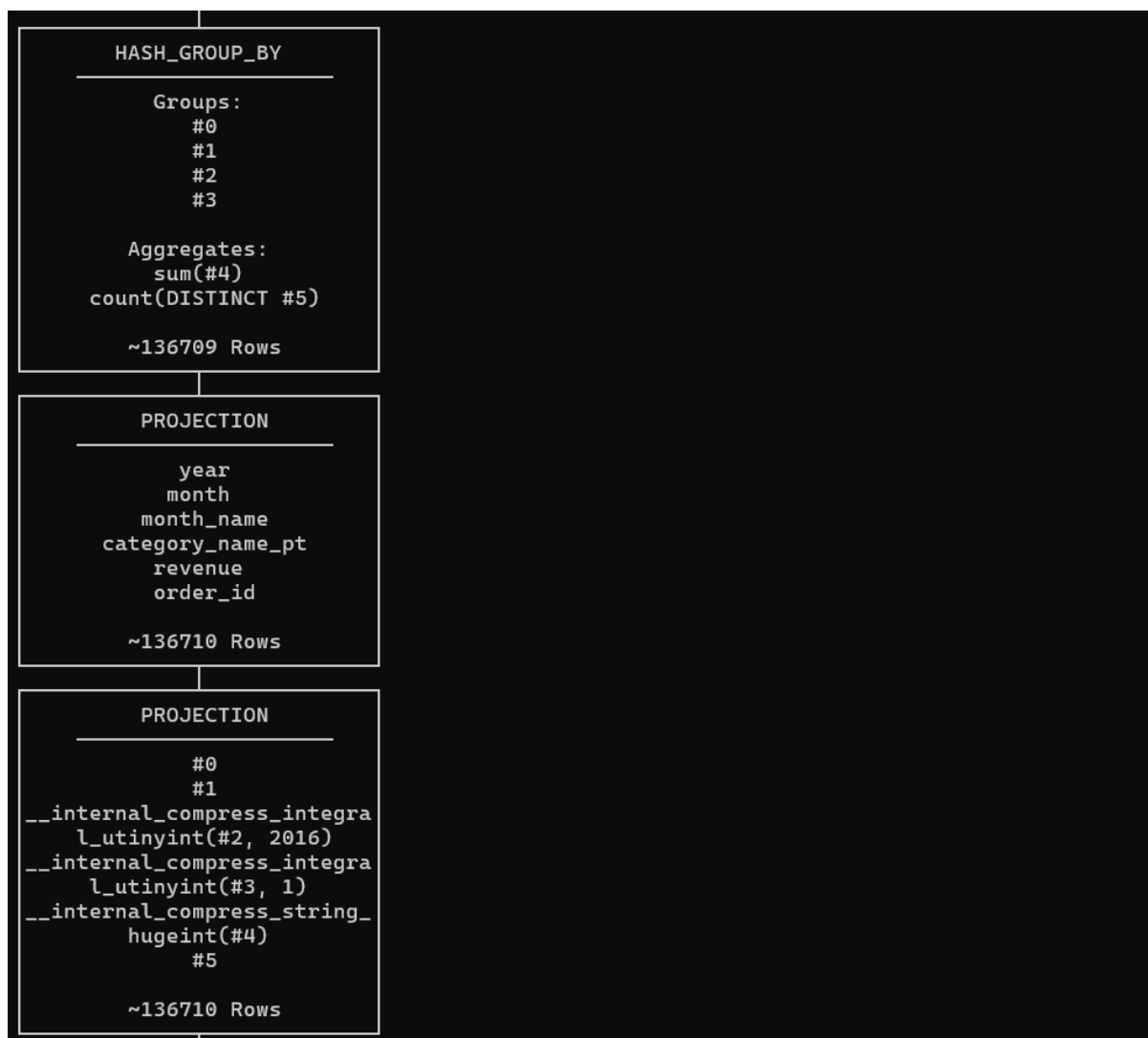
year
month_name
category_name_pt
receita_total
total_pedidos

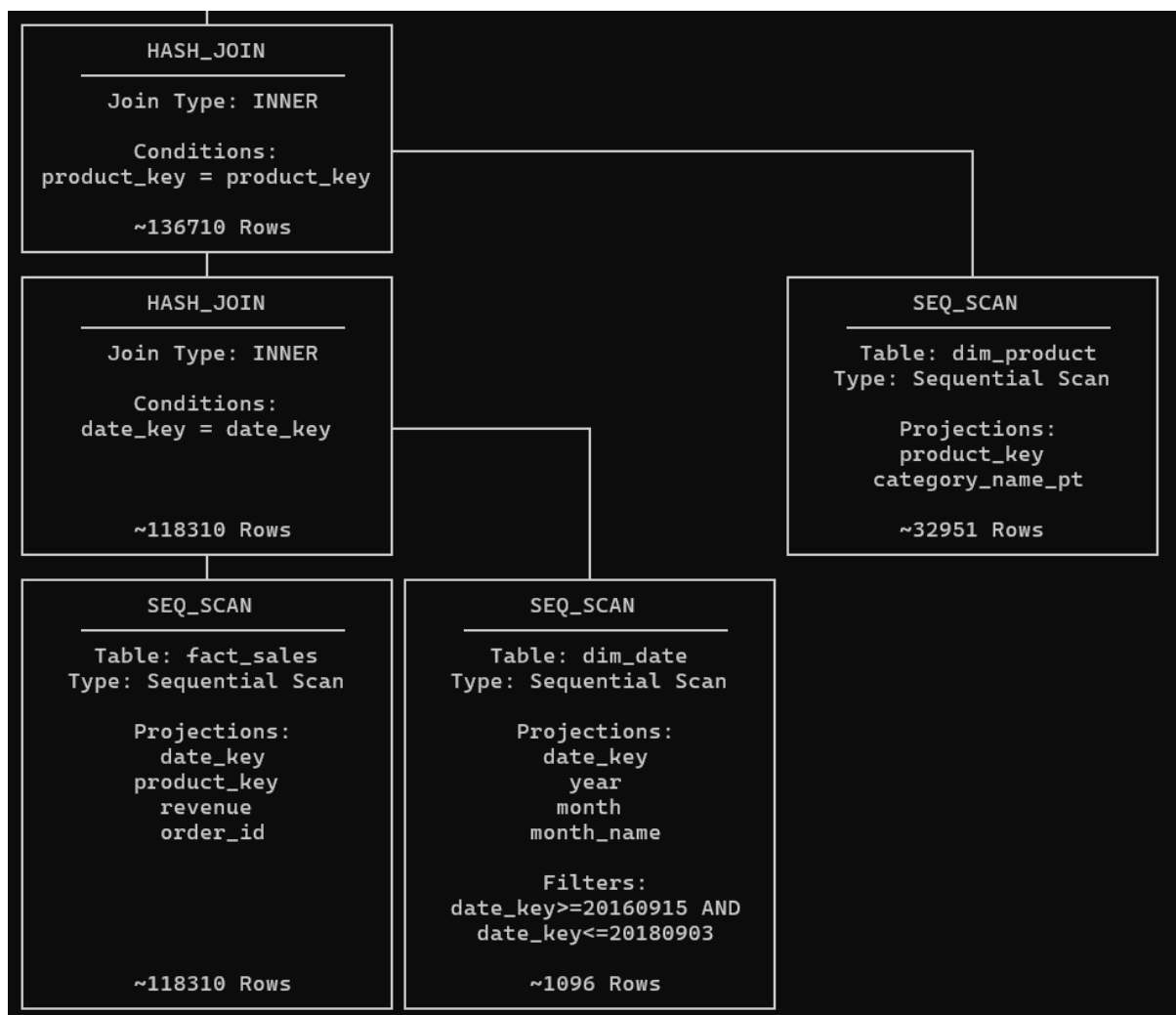
~136709 Rows

PROJECTION

--internal_decompress_integ
ral_integer(#0, 2016)
--internal_decompress_integ
ral_integer(#1, 1)
--internal_decompress_strin
g(#2)
#3
#4
#5

~136709 Rows





?? Analisando Plano de Execução na Tabela Agregada Otimizada...

Physical Plan

SEQ_SCAN

Table:
agg_monthly_sales_by_category

Type: Sequential Scan

Projections:
 year
 month_name
 category_name
 revenue
 num_orders

~1279 Rows

?? Comparativo de Volumetria Escaneada para o Relatório:

estrutura varchar	linhas_processadas int64
Tabela Fato Original	118310
Tabela Agregada Otimizada	1279

7. GitHub - README

Projeto_DW_Olist

Projeto de Data Warehouse usando DuckDB e dataset Olist para avaliação de Banco e Armazém de Dados

Data Warehouse E-Commerce Olist

Este repositório contém a implementação completa de um **Data Warehouse (DW)** utilizando a modelagem estrela (*Star Schema*) para análise das operações de vendas e logística do e-commerce **Olist**. Todo o pipeline de dados (Ingestão, Staging, OLTP e OLAP) foi construído utilizando **DuckDB** e **SQL**, complementado por scripts de visualização analítica em **Python**.

Estrutura do Projeto

O projeto está organizado na seguinte estrutura de diretórios e arquivos:

dw-olist/

```

├── data/
│   └── olist/          # Arquivos .csv originais extraídos do Kaggle
├── Graficos/          # Diretório de destino para os gráficos gerados (PNG/HTML)
├── scripts/
│   ├── 00_setup_duckdb.sql  # Criação do ambiente de Staging (Tabelas stg_)
│   ├── 01_oltp.sql         # Criação e normalização do ambiente relacional (Schema oltp)
│   ├── 02_dw_model.sql     # Criação das Dimensões e Fatos (Schema dw)
│   ├── 03_etl_load.sql     # Pipeline de Carga ETL e controle de histórico SCD Tipo 2
│   └── 05_perf.sql         # Scripts bônus de Otimização e Materialização (Performance)
├── demo.duckdb          # Banco de dados embarcado DuckDB (Gerado após
execução)
├── duckdb.exe           # Executável do CLI do DuckDB (Windows)
├── gerar_graficos.py     # Script Python para geração dos 4 gráficos obrigatórios
└── README.md            # Documentação do projeto

```

Pré-requisitos

Antes de iniciar a execução, certifique-se de ter os seguintes componentes configurados na sua máquina:

1. Baixar dataset brazilianecommerce

Acesse o site e baixe o dataset para a pasta /data/olist

Kaggle: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

2. Baixar dataset brazilian-ecommerce olist

Acesse o site e baixe o dataset para a pasta /data/olist

3. Verificar a Versão do Python

Abra o PowerShell e execute: Verificar se o Python está instalado

PowerShell python --version

Se abrir a Microsoft Store ao digitar python: Vá em: Configurações → Aplicativos → Aliases de execução do aplicativo Desative os aliases "python.exe" e "python3.exe"

Atenção: Microsoft Store Se o Windows abrir a Store ao digitar "python", desative o alias em Configurações → Aplicativos → Aliases de execução. Depois instale o Python pelo site oficial: python.org/downloads

4. Instalar o DuckDB CLI no Windows

- Colocar o executável duckdb.exe na pasta raiz do projeto.
- Testar o duckdb.exe local

PowerShell .\duckdb.exe demo.duckdb -c "SELECT 'ok' AS status;"

- Se preferir instalar globalmente (via winget):

PowerShell winget install DuckDB.cli

- Verificar versão

PowerShell duckdb --version

Nota: duckdb.exe local vs. global Se o duckdb não estiver no PATH, use sempre .\duckdb.exe (com ponto e barra) para chamar o executável local da pasta do projeto. O script run_all.ps1 detecta automaticamente qual usar.

5. Configurar a Política de Execução do PowerShell

- Liberar scripts para o seu usuário (execute uma única vez) Set-ExecutionPolicy - Scope CurrentUser RemoteSigned
- Se o Windows reclamar de assinatura, desbloqueie os arquivos:

PowerShell Unblock-File .\run_all.ps1

PowerShell Unblock-File .\scripts*.sql

- Alternativa temporária (sem alterar a política permanente):

powershell -ExecutionPolicy Bypass -File .\run_all.ps1

6. Criar o Ambiente Virtual Python

- Entrar na pasta do projeto

PowerShell cd C:\data\dw-atividade-olist-sample

- Criar ambiente virtual

PowerShell python -m venv .venv

- Ativar o ambiente (PowerShell)

PowerShell ..venv\Scripts\Activate.ps1

- Atualizar o pip e instalar as dependências

PowerShell python -m pip install --upgrade pip

PowerShell pip install --only-binary=:all: duckdb pandas plotly kaleido

- Verificar instalação

PowerShell python -c "import duckdb, pandas, plotly; print('OK')"

Nota: --only-binary=:all:

Este parâmetro instrui o pip a baixar apenas versões pré-compiladas (wheels), evitando tentativas de compilar o DuckDB localmente, o que poderia falhar por falta de compiladores C no Windows.

Passo a Passo para Execução do Pipeline

Abra o seu terminal (preferencialmente o PowerShell no Windows) na raiz do projeto (C:\Users...\data\dw-atividade-olist-sample) e execute os passos na ordem cronológica abaixo:

Passo 1: Ingestão e Carga da Camada de Staging

Este script realiza a leitura dos arquivos .csv brutos contidos na pasta data/Olist/ e os espelha para tabelas de staging temporárias de forma ultra rápida.

```
PowerShell .\duckdb.exe demo.duckdb -c ".read scripts/00_setup_duckdb.sql"
```

Passo 2: Construção da Camada Transacional (OLTP)

Lê as tabelas de staging, higieniza as nomenclaturas redundantes e tipa os dados estruturadamente no schema oltp.

```
PowerShell .\duckdb.exe demo.duckdb -c ".read scripts/01_oltp.sql"
```

Passo 3: Criação das Estruturas Multidimensionais do DW

Cria as tabelas físicas do modelo estrela (dim_date, dim_customer, dim_product, dim_seller, dim_geolocation e fact_sales) definindo as Chaves Primárias (PK) e Estrangeiras (FK).

```
PowerShell .\duckdb.exe demo.duckdb -c ".read scripts/02_dw_model.sql"
```

Passo 4: Execução do Pipeline ETL (Carga OLAP)

Popula o Data Warehouse aplicando regras de negócio essenciais, geração de chaves substitutas (Surrogate Keys) e tratamento de histórico usando Dimensões de Mudança Lenta (SCD Tipo 2).

```
PowerShell .\duckdb.exe demo.duckdb -c ".read scripts/03_etl_load.sql"
```

Passo 5: Execução do Script de Performance e Otimização (Bônus)

Cria uma tabela analítica agregada (Data Mart), gera índices secundários B-Tree e exibe o comparativo do plano físico (EXPLAIN) demonstrando uma redução drástica no volume de linhas escaneadas de mais de 118 mil linhas para pouco mais de 1.200 linhas.

```
PowerShell .\duckdb.exe demo.duckdb -c ".read scripts/05_perf.sql"
```

Executando tudo

Executa todos os 5 scripts do pipeline completo de uma só vez

```
PowerShell .\run_all.ps1
```

Geração dos Gráficos Analíticos

Com o banco de dados demo.duckdb totalmente carregado e otimizado através dos passos anteriores, execute o script Python para extrair as métricas e gerar os relatórios visuais obrigatórios:

```
PowerShell python gerar_graficos.py
```

Resultados Esperados da Execução:

O terminal exibirá um cabeçalho oficial com as estatísticas consolidadas do volume do dataset e logs de confirmação acompanhados de checkmarks (☑) para cada item gerado com sucesso.

Os seguintes arquivos de imagem e painéis serão criados de forma automática dentro do diretório /Graficos:

grafico_1_evolucao.png (Gráfico de Linha: Evolução Temporal de Faturamento)

grafico_2_top_produtos.png (Gráfico de Barras: Top 10 Categorias por Receita)

grafico_3_dispersao.png (Gráfico de Dispersão: Correlação entre Ticket Médio vs. Avaliação por Estado)

dashboard.html (Painel Executivo Analítico Interativo agrupando as visões em uma única composição de subplots)